

Number: P3931R0

Date: 2025-12-14

Audience: [Library Evolution](#)

Target: C++29

Authors: [Hana Dusíková](#) and [Jason Turner](#)

- [Motivation](#)
- [Implementation](#)
 - [No breakage](#)
 - [Debugability](#)
- [Wording](#)
 - [Feature test macro](#)

P3931R0: consteval all the non-allocating operators

This paper makes all non-allocating operator " " in the standard library marked `constexpr`. This doesn't break any code.

Motivation

Functions which don't have any runtime dependency and just creates an object should be `constexpr`, unless they have some implicit external dependency, then they should be only `constexpr`, so they are still usable in constant evaluation.

Implementation

Basically just a replacement of `constexpr` with `constexpr` in [my fork of libc++](#).

No breakage

A dear reader can think this change will break some code by making pointers to these functions `constexpr`-only values, but the standard already [disallow us from taking address of most functions](#) in the standard library, hence this is not a breakage.

Debugability

Any problematic code or a bug inside these functions will result in a compilation error instead runtime bug which needs to be debugged. This is seen as strict improvement, and even so users tends to prefer to skip over `std::` namespaced functions when looking for bugs.

Wording

Note: only operator which is not changed is `std::basic_string` operators `"s` as we still don't have non-transient allocation out of constant evaluation to runtime.

27.3.7 Suffix for `basic_string_view` literals [string.view.literals]

```
constexpr consteval string_view operator""sv(const char* str, size_t len) noexcept;
```

1 Returns: `string_view{str, len}`.

```
constexpr consteval u8string_view operator""sv(const char8_t* str, size_t len) noexcept;
```

2 Returns: `u8string_view{str, len}`.

```
constexpr consteval u16string_view operator""sv(const char16_t* str, size_t len) noexcept;
```

3 Returns: `u16string_view{str, len}`.

```
constexpr consteval u32string_view operator""sv(const char32_t* str, size_t len) noexcept;
```

4 Returns: `u32string_view{str, len}`.

```
constexpr consteval wstring_view operator""sv(const wchar_t* str, size_t len) noexcept;
```

5 Returns: `wstring_view{str, len}`.

29.4.11 Suffixes for complex number literals [complex.literals]

1 This subclause describes literal suffixes for constructing complex number literals. The suffixes `i`, `il`, and `if` create complex numbers of the types `complex<double>`, `complex<long double>`, and `complex<float>` respectively, with their imaginary part denoted by the given literal number and the real part being zero.

```
constexpr consteval complex<long double> operator""il(long double d);  
constexpr consteval complex<long double> operator""il(unsigned long long d);
```

2 Returns: `complex<long double>{0.0L, static_cast<long double>(d)}`.

```
constexpr consteval complex<double> operator""i(long double d);  
constexpr consteval complex<double> operator""i(unsigned long long d);
```

3 Returns: `complex<double>{0.0, static_cast<double>(d)}`.

```
constexpr consteval complex<float> operator""if(long double d);
constexpr consteval complex<float> operator""if(unsigned long long d);
```

4 *Returns:* `complex<float>{0.0f, static_cast<float>(d)}`.

30.5.9 Suffixes for duration literals [time.duration.literals]

- 1 This subclause describes literal suffixes for constructing duration literals. The suffixes `h`, `min`, `s`, `ms`, `us`, `ns` denote duration values of the corresponding types hours, minutes, seconds, milliseconds, microseconds, and nanoseconds respectively if they are applied to *integer-literals*.
- 2 If any of these suffixes are applied to a *floating-point-literal* the result is a `chrono::duration` literal with an unspecified floating-point representation.
- 3 If any of these suffixes are applied to an *integer-literal* and the resulting `chrono::duration` value cannot be represented in the result type because of overflow, the program is ill-formed.

4 [Example 1: The following code shows some duration literals.

```
using namespace std::chrono_literals;
auto constexpr aday=24h;
auto constexpr lesson=45min;
auto constexpr halfanhour=0.5h;
```

— end example]

```
constexpr consteval chrono::hours operator""h(unsigned long long hours);
constexpr consteval chrono::duration<unspecified, ratio<3600, 1>> operator""h(long double hours);
```

5 *Returns:* A duration literal representing hours hours.

```
constexpr consteval chrono::minutes operator""min(unsigned long long minutes);
constexpr consteval chrono::duration<unspecified, ratio<60, 1>> operator""min(long double minutes);
```

6 *Returns:* A duration literal representing minutes minutes.

```
constexpr consteval chrono::seconds operator""s(unsigned long long sec);
constexpr consteval chrono::duration<unspecified> operator""s(long double sec);
```

7 *Returns:* A duration literal representing sec seconds.

8

[Note 1: The same suffix `s` is used for `basic_string` but there is no conflict, since duration suffixes apply to numbers and string literal suffixes apply to character array literals. — *end note*]

```
constexpr consteval chrono::milliseconds operator""ms(unsigned long long msec);
constexpr consteval chrono::duration<unspecified, milli> operator""ms(long double msec);
```

9 *Returns:* A duration literal representing msec milliseconds.

```
constexpr consteval chrono::microseconds operator""us(unsigned long long usec);
constexpr consteval chrono::duration<unspecified, micro> operator""us(long double usec);
```

10 *Returns:* A duration literal representing usec microseconds.

```
constexpr consteval chrono::nanoseconds operator""ns(unsigned long long nsec);
constexpr consteval chrono::duration<unspecified, nano> operator""ns(long double nsec);
```

11 *Returns:* A duration literal representing nsec nanoseconds.

30.8.3.3 Non-member functions [time.cal.day.nonmembers]

```
constexpr consteval chrono::day operator""d(unsigned long long d) noexcept;
```

10 *Returns:* `day{static_cast<unsigned>(d)}`.

30.8.5.3 Non-member functions [time.cal.year.nonmembers]

```
constexpr consteval chrono::year operator""y(unsigned long long y) noexcept;
```

10 *Returns:* `year{static_cast<int>(y)}`.

Feature test macro

17.3.2 Header `<version>` synopsis [version.syn]

```
#define __cpp_lib_consteval_literals 20????L
```